Cloud FinOps Multi-Agent Architecture

Author: Claus Albuquerque

Course: Agentic AI Program: Building Autonomous Systems for Real-World Applications

Date: August 23, 2026.

Problem Statement and the Multi-Agent Approach

Our system automates Cloud Financial Operations (FinOps) by identifying wasted cloud infrastructure spend and recommending cost-saving actions. This problem inherently involves conflicting objectives: aggressively minimizing financial cost versus safely maintaining infrastructure reliability and capacity headroom. A single-agent approach struggles to balance these competing concerns, often hallucinating unsafe rightsizing recommendations or ignoring complex infrastructure dependencies.

A multi-agent system solves this by physically separating financial optimization from engineering safety. We employ a "separation of duties" pattern, ensuring that cost-cutting proposals are rigorously audited by a reliability expert before reaching the human operator.

Agent Roles and Responsibilities

The system is intentionally constrained to **two specialized agents**. This binary design minimizes coordination overhead while enforcing the necessary adversarial balance between cost and reliability.

1. **FinOps Specialist Agent**:
    - **Role**: Financial optimizer and anomaly detective.
    - **Responsibilities**: Analyzes the normalized FOCUS dataset, queries time-series cost trends, detects spending anomalies, checks commitment discount coverage, and proposes initial optimization recommendations.
2. **SRE Specialist Agent**:
    - **Role**: Infrastructure safety validator and topology expert.
    - **Responsibilities**: Retrieves CPU and memory utilization telemetry, queries resource dependencies (e.g., attached storage, load balancers), and interprets workload baselines (e.g., suppressing alerts for expected batch jobs). Its primary job is to veto or approve the FinOps agent's proposals based on engineering constraints.

Coordination, ReAct Loop, and Communication Strategy

The workflow follows a **hybrid sequential-delegation strategy**. The process resembles a gated project network:

1. The orchestrator triggers the FinOps Agent to investigate a cost spike or inefficiency.
2. **ReAct Trace & Stop Conditions**: Both agents utilize grounded ReAct (Reasoning and Acting) loops. We enforce explicit stop conditions: the FinOps agent must halt its investigation loop the moment a root cause is isolated and a safe proposal is drafted, preventing infinite analytical spirals.
3. **Communication**: Instead of open-ended conversational debate, the agents exchange information via strict, one-to-one tool delegation (`delegate_to_sre` and `delegate_to_finops`).
4. The SRE Agent receives the proposal, executes its telemetry ReAct loop (halting as soon as infrastructure headroom constraints are verified), and replies with a definitive safety ruling (two-way

validation).
5. Only if the SRE Agent approves does the system render the final recommendation for the Human-in-the-Loop.

Design Decisions and Trade-offs

In designing this architecture, several key trade-offs were considered:

- **Reliability vs. Latency**: Adding the SRE validation loop significantly increases system reliability, preventing catastrophic production outages caused by naive downsizing. The trade-off is higher latency, as two separate ReAct reasoning loops must execute sequentially.
- **Complexity vs. Autonomy**: We explicitly disabled open-ended brainstorming between agents. While free-form debate can explore multiple paths, it introduces unpredictable coordination overhead and context bloat. By forcing communication through typed delegation tools, we trade some creative autonomy for deterministic, assembly-line efficiency.
- **Diminishing Returns**: We decided against adding a third "Planner" agent. Given the well-defined nature of cloud optimization, we utilize CrewAI's deterministic Flow orchestration rather than an LLM-based planner. CrewAI Flows allow us to define rigid, state-driven execution pipelines where tasks transition predictably from the FinOps agent to the SRE agent. This hybrid approach leverages the deterministic reliability of state machines for high-level coordination while preserving the autonomous, ReAct-based reasoning of the individual agents, entirely avoiding the latency and unpredictability of a dynamic AI planner.

Conclusion

This two-agent architecture supports effective problem-solving by mirroring real-world organizational structures (Finance vs. Engineering). It scales efficiently because the financial reasoning capabilities and the infrastructure telemetry tools are strictly decoupled, allowing independent refinement while guaranteeing that no cost-saving measure compromises system uptime.